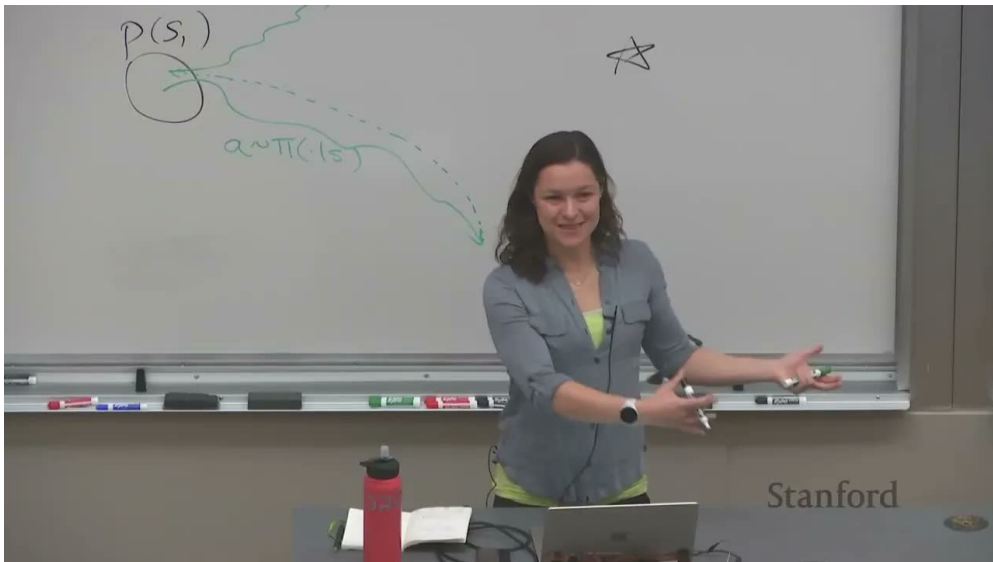


课程笔记

CS224R 课程笔记: Lecture 16 RL for Robots

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 讲次日期 2025-05-23; YouTube 上传 2025-12-08

视频时长: 1:05:44

视频链接: <https://www.youtube.com/watch?v=rbaWQQLrz10>

目录

1 问题背景：为什么机器人还没有实现真正自主学习	2
1.1 自主 RL 的定义	2
1.2 评价指标也随之改变	3
1.3 本章小结	3
2 长时程自主学习的难点	4
2.1 为什么简单地把 episode 拉长并不够	4
2.2 为何 episodic RL 能工作，而 reset-free RL 更难	4
2.3 先学会回去，才有机会继续学会向前	4
2.4 本章小结	5
3 核心方法：学习重置、学习循环、学习恢复	5
3.1 Forward-Backward RL	5
3.2 从“回到起点”到“回到专家状态”	6
3.3 任务循环：不只做前进和后退	6
3.4 本章小结	7
4 Single-Life RL：部署时也要在单次生命里适应	7
4.1 问题设定	7
4.2 为什么直接 test-time fine-tuning 不够	7
4.3 两个更有前景的方向	8
4.4 本章小结	8
5 总结与延伸	8

1 问题背景：为什么机器人还没有实现真正自主学习

这讲围绕一个非常具体、但常被标准 RL 设定掩盖的问题展开：机器人当然可以在 rollout 期间根据当前策略自主执行动作，但一旦任务失败或偏离起点，究竟是谁把它放回可以重新尝试的位置？在仿真里，这一步被一句 `env.reset()` 掩盖；在真实世界里，这一步往往意味着人类重新摆放物体、扶正机器人、关上门、把毛巾展开，或者把移动平台重新开回起点。

Standard Reinforcement Learning

$s_0, a_0, s_1, a_1 \dots s_H$

$s'_0, a'_0, s'_1, a'_1 \dots$

How does this happen?

```
import gym
env = gym.make("CartPole-v1")
observation = env.reset()
for _ in range(1000):
    env.render()
    action = env.action_space.sample() # you can
    observation, reward, done, info = env.step(action)
    if done:
        observation = env.reset()
env.close()
```

[Code snippet from <https://gym.openai.com/>]

Only in simulation!

Slide adapted from Archit Sharma

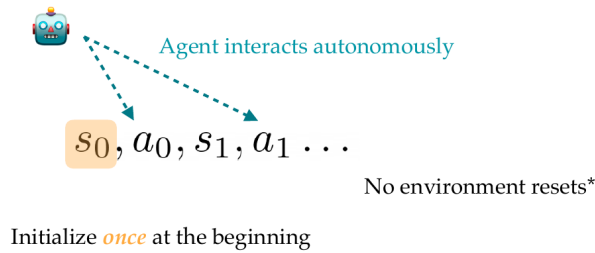
图 1：标准强化学习默认存在环境重置接口；这一前提在真实机器人场景中往往不成立。

讲者把这一点表述得很直接：传统 RL 并不是完全自主的，只是把“自主”限定在一次 episode 内。机器人学里真正稀缺的不是动作执行能力，而是长时间、无人工干预的练习能力。对于导航、抓取、开门、整理物体等任务，能够反复自主尝试，比单次成功更接近长期部署时真正需要的能力。

1.1 自主 RL 的定义

本讲把 autonomous RL 定义为一种更严格的交互模式：环境在开始时初始化一次，之后智能体持续与世界交互，不依赖频繁环境重置。这个定义允许偶尔低频 reset，但关键思想是不再把 reset 当成免费操作。

Autonomous RL: Definition



Slide adapted from Archit Sharma

*can relax this constraint to reset occasionally, or at low frequency

图 2: 自主 RL 的核心要求: 初始化一次, 然后在尽量没有环境重置的前提下持续学习。

这个定义把“训练时 autonomy”放到和“部署时 autonomy”同样重要的位置。讲者特别强调, 机器人不仅要在任务执行时减少人类介入, 也要在训练过程中减少人类介入, 因为只有这样, 真实世界数据规模才可能显著扩大, 而数据规模扩大又可能带来更好的泛化和鲁棒性。

1.2 评价指标也随之改变

一旦去掉频繁 reset, 评价标准也不再只有“最后学到的策略有多好”。本讲区分了两种目标:

- **deployed policy evaluation**: 最终学到的策略在标准起点上有多强, 适合关心最终性能的任务, 例如机器人厨师。
- **continuing policy evaluation**: 智能体在整段生命期里累计拿到了多少奖励, 适合关心持续产出的任务, 例如火星车。

这两个目标都合理, 但对应不同算法偏好。前者允许训练时为“最终好策略”忍受很多弯路; 后者则要求学习过程本身就尽量少犯代价高昂的错误。

本讲的关键转向

把真实世界 RL 看成“单次 episode 内优化动作”是不够的。更准确的视角是: 智能体需要在很长的生命期中, 一边探索、一边恢复、一边继续完成任务。

1.3 本章小结

这一部分把问题重新摆正了: 机器人 RL 的核心瓶颈并不只是策略优化, 而是如何在没有人类频繁重置环境的前提下持续训练。自主 RL 的定义和评价方式都因此发生变化。

2 长时程自主学习的难点

2.1 为什么简单地把 episode 拉长并不够

一个自然想法是：既然 reset 代价高，那就把 horizon H 拉长，让 agent 少 reset。讲者指出，这样做虽然减少了监督频率，却不会自动得到好结果。原因在于标准 RL 在长时程、非回合式训练里会同时遇到两个问题。

The Challenge of Learning Autonomously

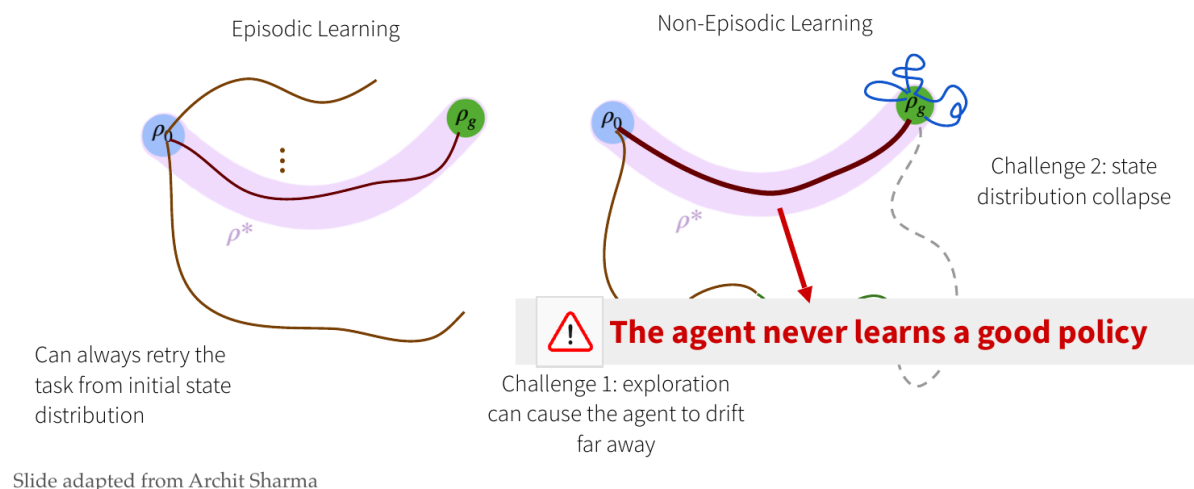


图 3: 非回合式训练的两个核心挑战：探索让轨迹漂移离开任务分布，状态分布坍塌让 agent 学不到好策略。

第一，**exploration drift**。智能体在探索时很容易漂移到与任务无关、甚至难以恢复的状态区域。第二，**state distribution collapse**。当智能体离开有意义的状态簇之后，后续采样越来越集中在“坏状态”附近，更新也会围绕这些坏状态展开，最后甚至学不到有效策略。

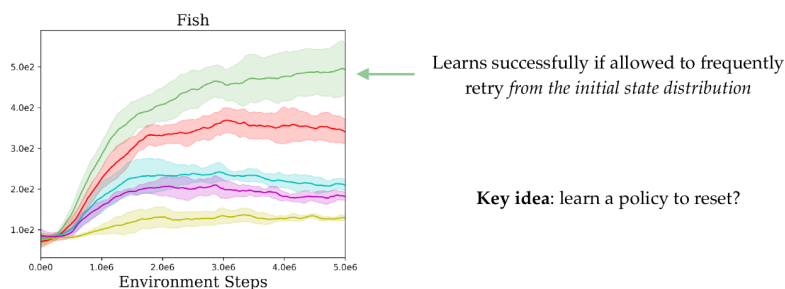
2.2 为何 episodic RL 能工作，而 reset-free RL 更难

在标准 episodic setting 中，失败一次并不严重，因为下一次又会从初始状态分布 ρ_0 重新开始。这样训练数据天然围绕“可完成任务”的状态区域展开，学习信号始终比较集中。到了 non-episodic setting，这种隐式课程设计消失了。失败之后，agent 还得自己想办法回到有用状态，而这件事本身就是一个新的控制任务。

2.3 先学会回去，才有机会继续学会向前

讲者接着展示了一个经验现象：如果允许算法频繁回到初始状态，它能学得很好；一旦去掉频繁 reset，性能显著下降。这说明问题不在于 forward task 本身太难，而在于 agent 缺乏回到“可学习区域”的机制。

How to autonomously learn an effective policy?



Slide adapted from Archit Sharma

图 4: 实验趋势说明: 能够频繁回到初始分布时, forward task 能学会, 因此关键难点转化为“如何学会恢复或重置”。

这正是本讲后续算法设计的出发点。比起一味提高探索强度, 更有效的路线是显式学习恢复能力。

2.4 本章小结

单纯增加 horizon 不能解决真实世界自主训练问题。真正困难的是, agent 在失败后会漂移到陌生且难恢复的状态, 从而让训练分布逐步劣化。因此, 自主 RL 需要把“恢复”“回退”“重置”本身纳入学习对象。

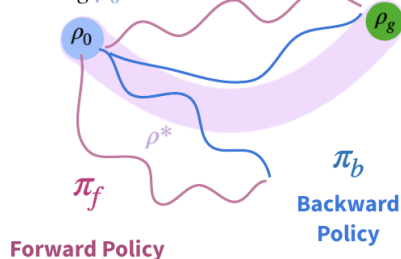
3 核心方法: 学习重置、学习循环、学习恢复

3.1 Forward-Backward RL

最核心的思路是: 为任务同时学习一个前向策略 π_f 和一个后向策略 π_b 。前向策略负责完成任务, 后向策略负责回到接近初始分布的状态。两者交替 rollout、各自按对应奖励更新, 从而减少对人工 reset 的依赖。

Algorithm: Forward-Backward RL

Define $r_b(s, a)$
for reaching ρ_0



Forward-Backward RL (FBRL)

1. Initialize forward policy π_f and backward policy π_b
2. Repeat:
 1. Roll out π_f for H steps
 2. Update π_f using $r_f(s, a)$
 3. Roll out π_b for H steps
 4. Update π_b using $r_b(s, a)$
 5. No reset or very infrequent reset

Test time: Discard π_b . Deploy π_f

+ simple - backward policy is ultimately discarded

[Han et al. Learning Compound Multi-Step Controllers under Unknown Dynamics]
[Eysenbach et al. Leave no Trace: Learning to Reset for Safe and Autonomous Reinforcement Learning]

图 5: Forward-Backward RL 的基本流程：前向策略做任务，后向策略学习回到初始区域；测试时只保留前向策略。

可以把它写成下面这个交替过程：

rollout $\pi_f \rightarrow$ update $\pi_f \rightarrow$ rollout $\pi_b \rightarrow$ update π_b .

其中：

- π_f 的奖励 $r_f(s, a)$ 鼓励到达目标区域；
- π_b 的奖励 $r_b(s, a)$ 鼓励回到起始状态分布附近；
- 训练时尽量不 reset，只在必要时低频 reset；
- 测试时丢弃 π_b ，只部署 π_f 。

这个方法的优点是概念简单，而且能直接把“恢复能力”做成一个可学习对象；缺点也很明确：后向策略在最终部署时会被丢弃，因此训练花费的一部分能力并不直接服务于最终任务。

3.2 从“回到起点”到“回到专家状态”

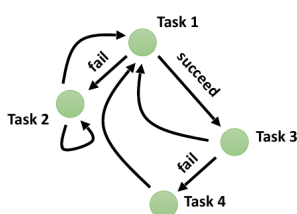
讲者随后介绍了一个重要变体：后向策略不一定非得回到单一初始状态，而可以回到示范轨迹或专家轨迹覆盖到的状态分布。这样做的动机是，真实任务往往存在一大片“有用但不完全等于起点”的恢复区域。只要能回到这些熟悉状态，forward policy 就可能继续完成任务。

这个修改本质上把 backward reward 从“reset to start”推广为“reset to good familiar states”，既减轻了恢复任务难度，也更适合用 demonstration 数据做辅助监督。

3.3 任务循环：不只做前进和后退

如果环境天然支持多个相互连接的子任务，那么更一般的做法不是一味学一前一后两种行为，而是构造任务循环，让 agent 在一个任务图中不断切换练习对象。

Algorithm: Autonomous RL with task cycles



1. Initialize task-conditioned policy $\pi(\cdot | \mathbf{s}, \mathbf{z})$
2. Repeat:
 1. Propose task $\mathbf{z}_i$ to practice based on $\mathbf{s}$
 2. Roll out $\pi(\cdot | \mathbf{s}, \mathbf{z}_i)$ for H steps
 3. Update π using $r(\mathbf{s}, \mathbf{a}, \mathbf{z})$
 4. No reset or very infrequent reset

Test time: Deploy π

图 6: 任务循环视角: 把环境中的恢复、前进、切换都看成任务图上的边, 从而避免把恢复限制成单一 backward task。

这比单纯的 forward-backward 更一般。比如开门任务可以和关门任务组成循环; 拾起杯子、放回杯子、移动到新位置也可以组成任务链。训练时 agent 根据当前状态选择合适任务, 而不是固定执行“做任务”或“撤回任务”。

从算法设计角度看

本讲的共识是: 真实世界自主训练需要显式建模可恢复性。恢复可以是 backward policy、expert-state reset policy, 或者更一般的 task graph。

3.4 本章小结

reset-free RL 的关键不是更激进地探索, 而是学习怎样回到有用状态。Forward-Backward RL 提供了最直接的框架, 而 expert-state reset 和 task-cycle 则把恢复从“回到起点”推广为“回到可继续学习的熟悉状态”。

4 Single-Life RL: 部署时也要在单次生命里适应

4.1 问题设定

讲座最后把问题进一步推进到更困难的 single-life setting: 智能体并不是从零开始训练, 而是已经有过预训练经验; 现在它进入新环境, 只有“一条命”, 要在单次部署过程中一边执行、一边适应, 不能依赖 reset 反复试错。

这和前面的 autonomous RL 有联系, 但更强调部署时在线适应。讲者用人开门失败后迅速调整钥匙位置的例子来说明, 人类在部署中也会即时纠错, 而不是回到训练室重新训练。

4.2 为什么直接 test-time fine-tuning 不够

一个简单基线是: 部署时继续对原策略做 TD update 或 policy gradient update。讲者展示的现象是, 这种做法有时能临时学会越过一个障碍, 但一旦进入此前没见过的卡死状态, 就可能再也

爬不出来。原因在于：

- 在线更新仍然围绕当前目标奖励进行，未必鼓励先恢复到熟悉状态；
- 低层动作空间中的微调幅度有限，很难实现策略层面的快速改道；
- 进入坏状态后，局部数据分布太差，更新反而容易越调越偏。

4.3 两个更有前景的方向

讲者给出两类更有希望的思路。第一类，是在部署时把 RL 信号改成“回到熟悉状态”，而不是直接“继续冲向目标”。这样 agent 先把自己从坏状态拉回训练分布附近，再继续完成任务。第二类，是在更高层的技能空间里适应，而不是只在低层动作上微调。例如预训练“向前走”“翻身站起”“重新接近门把手”等技能，部署时在技能层面切换，会比逐动作修补更有效。

这类方法与现代 foundation model 也自然衔接：如果预训练模型本身带有较强的常识和技能先验，那么 single-life adaptation 可以少一些盲目试错，多一些结构化恢复。

4.4 本章小结

single-life RL 讨论的是更贴近真实部署的问题：系统已经学过一些东西，但在陌生条件下只有一次机会。此时仅靠低层在线微调通常不够，更关键的是恢复到熟悉状态以及调用更高层、可组合的技能。

5 总结与延伸

Lecture 16 的价值在于，它把真实机器人 RL 中最常被忽略的成本显式化了：**reset 不是免费的**。一旦接受这一点，问题就从“怎么学会一个好策略”扩展成“怎么在真实世界里持续地学、恢复、再继续学”。这也是为什么 autonomous RL 既关心最终策略质量，也关心生命期累计回报。

从方法上看，本讲给出了一条非常清晰的谱系：

- 最基础的是 Forward-Backward RL，用后向策略显式学习恢复。
- 更实用的是把恢复目标放宽到示范覆盖的熟悉状态，而不是单一起点。
- 更一般的是 task cycles，把恢复、重试和子任务切换统一进任务图。
- 更前沿的是 single-life RL，要求系统在单次部署里继续适应。

如果把这讲和后续机器人课程线索连起来，可以得到一个很强的研究命题：真实世界 RL 的核心并不只是 sample efficiency，而是 **recoverability + adaptability**。未来更值得关注的方向包括如何自动发现恢复技能、如何用 demonstrations 或 world models 定义熟悉状态、以及如何把 LLM 或技能库用于单生命部署中的快速决策切换。